AI-Assisted Coding - Mid-Year Project

Malda. O Tabbah
American University of Beirut
AI-Assisted Coding SU25-26
Ammar Mohanna
16/08/2026

.

Background

The task tracker consists of a FastAPI backend with in-memory storage and a single-page Kanban frontend that communicates with it via HTTP APIs. The interface contains three columns: To do, In progress, and Done. Each card within a column consists of a title, a priority label, an option description, and an optional assignee. Tasks are automatically stored by priority. A task counter updates live.

Data Model

Each task has an ID, Title, Description, Status (ToDo, InProgress, Done) , Priority (`Low` / `Medium` / `High`), optional assignee, and timestamps.

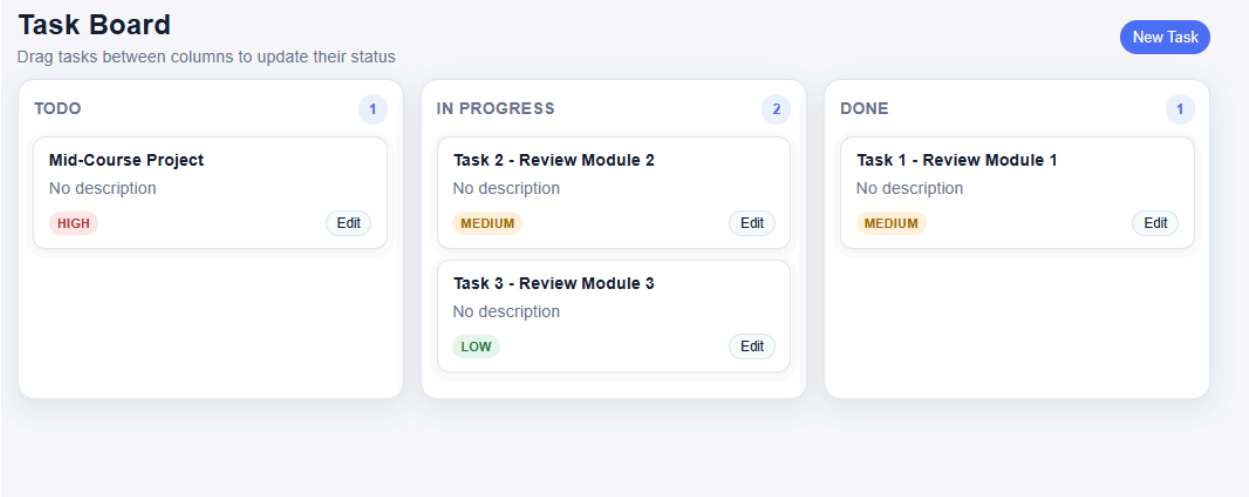
Backend

The API (`http://localhost:8000`) supports:

- `GET /health` and `GET /version`
- List tasks (optional `status` / `priority` filters), get by id, create, partial update (`PATCH`), and delete

Frontend

The UI (`frontend/index.html`) is a three-column board with loading, ready, empty, and error states. You can create/edit tasks in a form and drag cards between columns. The client checks the same transition rules before calling `PATCH`. The UI does not expose delete, health, or version.



Business Rule

- Title is required, non-blank after strip, max 200 characters; unknown fields are rejected.

- New tasks default to `ToDo`, `Medium` priority, empty description.

- Allowed status changes: **`ToDo → InProgress`**, **`InProgress → Done`**, **`Done → InProgress`**. Same status is a no-op. Anything else returns 422 with the allowed transitions listed.

- Rules are enforced on the server (`business_rules.py`) and mirrored in the frontend.

Objective

In this project, we will extend the task tracker functionality by adding the following features:

Feature Option	Expected Backend Work	Expected Frontend Work	Good tests to include
Due Date + Overdue Filter	- Add option due_date validation. - Decide whether overdue is computed in the backend and UI. - Optional query filter for overdue	- Add due date to the modal form - Show due date or overdue pill on cards. - Add an overdue filter or visual indicator	- Valid due date - Invalid date format - Overdue date detection - Update due date - Filter returns only overdue tasks
Search + Combined filters	- Extend GET /tasks to support text search and combinations such as status, priority, assignee, or tag/due-date if implemented	- Add a compact filter/search bar above the board. Keep columns visible and preserve empty states	Search title/description, combined status + priority, no matches

Functional Requirement Specification

The first step would be generating the user stories for the features above:

Prompt for Creating User Stories

AI Tool Used: ChatGPT Pro
Model: GPT 5.5

Role: You are a senior product owner who wants to extend the Task Tracker web application with the following additional features:

Included Features:

- The task has a due date and due date change date as additional attributes.
- The task becomes overdue if the due date has passed and the status is still 'ToDo' or 'In Progress'
- The task due date can be changed.
- When the task due date is changed, the due date change date is updated with the current system date automatically.
- Tasks can be filtered and searched by due dates, an overdue flag check, status, priority, assignee, and task name.

Explicitly out of scope:

- authentication
- user accounts
- multi-tenancy or per-user task lists
- real-time updates
- mobile app
- notifications
- production database or deployment in Module 1

Target user: A solo developer or small team managing work in a single shared task list.

Task: Generate 3-4 user stories in the format: As a [role], I want [feature] so that [benefit].

Constraints:

- Use "team member" as the main role unless another role is clearly needed.
- For each story, include 4-5 acceptance criteria that are specific and testable.
- Cover happy paths and at least one failure case across the set.
- Do not add features outside the scope above.

Output format: Return a table with columns: ID, Story, Acceptance Criteria, Notes / Assumptions

AI Response

ID	Story	Acceptance Criteria	Notes / Assumptions
US-08	As a team member, I want to assign a due date to a task so I know when it is expected to be completed.	1. A due date can be provided when creating a task. 2. The due date is stored and displayed when the task is viewed. 3. The due date must be a valid calendar date; an invalid date format is rejected with a validation error. 4. If no due date is provided, the task is created without a due date. 5. The due date change date is empty when the due date has never been modified after task creation.	- Description remains optional.- Due date is optional unless future business rules require otherwise. - Creating a task with an initial due date does **not** count as a due date change.

US-09	As a team member, I want to update a task's due date so that I can keep the schedule accurate when priorities or timelines change.	1. A task's due date can be updated after the task has been created. 2. When the due date is successfully changed, the due date change date is automatically set to the current system date. 3. Updating other task attributes without changing the due date does not modify the due date change date. 4. An invalid due date value is rejected with a validation error, and neither the due date nor the due date change date is updated. 5. Attempting to update the due date of a non-existent task returns a "Task not found" error.	- The due date change date is system-generated and cannot be manually supplied by the user.- "Current system date" refers to the server date.
US-10	As a team member, I want the system to identify overdue tasks so that I can quickly focus on work requiring immediate attention.	1. A task is considered overdue when the current system date is later than its due date and its status is **ToDo** or **In Progress**. 2. A task with status **Done** is never marked as overdue, regardless of its due date. 3. A task with no due date is not considered overdue. 4. When the status changes to **Done**, the overdue flag is no longer returned. 5. If the due date is today, the task is not considered overdue until the due date has passed according to the application's date comparison rule.	- The overdue flag is derived from existing data and is not stored separately. - Date comparison uses the server's system date.
US-11	As a team member, I want to search and filter tasks using task details and due-date information so that I can quickly locate relevant work.	1. Tasks can be filtered by due date. 2. Tasks can be filtered by overdue status (overdue or not overdue). 3. Tasks can be filtered by status, priority, and assignee. 4. Tasks can be searched by task title (task name). 5. If no tasks match the selected search or filter criteria, the system displays an empty result without an error.	- Multiple filters may be applied together using logical **AND**. - Task name search is assumed to be a partial, case-insensitive match unless otherwise specified.

Inspection and Review

US-08 Review

During inspection, US-08 assumes the due date is optional, but we want to specify that a due date at task creation is mandatory. Thus, the following changes are reviewed for US-08

AI Generated	Human Inspected
Acceptance Criteria 4. If no due date is provided, the task is created without a due date.	**Acceptance Criteria** 4. If no due date is provided, the task is not created, and an error message indicates that a due date is required.
Notes / Assumptions - Description remains optional.- Due date is optional unless future business rules require otherwise.	**Notes / Assumptions** - Description remains optional.- Due date is mandatory.

US-10 Review

During inspection, US-10 assumes that if the task is done, the overdue flag is no longer returned. We need this to change so that the overdue flag is automatically set off when the task's status changes to Done.

AI Generated	Human Inspected
Acceptance Criteria 4. When the status changes to Done, the overdue flag is no longer returned	**Acceptance Criteria** 4. When the status changes to Done, the overdue flag is set off automatically.

Prompt for Generating User Stories Documentation

You are a technical product owner working inside an existing Task Tracker web application project.

Task: Create a Markdown documentation file at: docs/user_stories.md
If the `docs` folder does not exist, create it.

Constraints:
- Do not generate new user stories.
- Use only the user stories already provided and reviewed below. Preserve the story IDs, story wording, acceptance criteria, and notes/assumptions as much as possible.
- You may only improve the Markdown structure, formatting, numbering, and readability.
- Do not add features, acceptance criteria, or assumptions that are not already present in the provided stories.

Context:
The Task Tracker is a learning project with:
- Python/FastAPI backend
- Pydantic validation
- Simple HTML/CSS/JavaScript frontend
- One shared task list for a solo developer or small team
- The project is not intended to be production software.

Existing Scope:
The application supports:
- Create tasks
- View all tasks
- Filter tasks by status and priority
- Update task details
- Update task status using a forward-only workflow
- Delete tasks

Extension Scope:
The Task Tracker is being extended with:
- Required due date
- Due date change date
- Overdue task detection
- Ability to update the due date
- Automatic update of due date change date when due date changes
- Search and filtering by due date, overdue flag, status, priority, assignee, and task name

Explicitly Out of Scope:
Do not include or suggest anything related to:
- Authentication
- User accounts
- Multi-tenancy
- Per-user task lists
- Real-time updates
- Mobile app
- Notifications
- Microservices
- Docker
- Cloud deployment
- Production database setup

Provided User Stories
Use the following user stories as the source of truth:
[PASTE USER STORIES USER STORIES THAT ARE INSPECTED AND REVIEWED]

AI Response
See project folder /task-tracker - mid/docs/user_stories.md

Technical Requirement Specification
The second step is to review the data model and technical stack for this extension.

AI Tool Used: ChatGPT Pro
Model: GPT 5.5

Prompt for Creating a Mini-ADR

Role: You are a technical product owner and a software architect who wants to propose changes in the architecture design record of the existing task tracker application

Context: The existing task tracker consists of a Python FastAPI backend and a simple web frontend. We are extending the task tracker's current features to include the following requirements.

>>>>>
[PASTE USER STORIES USER STORIES THAT ARE INSPECTED AND REVIEWED]
<<<<<

Task: Review the original task-tracker ADR and generate a new mini-ADR that has the same headings, indicating the changes that are needed (if any) for the requirements to be incorporated. If the existing architecture fits the new requirement, please indicate that.

>>>>>
[PASTE THE ADR GENERATED AND APPROVED IN MODULE 1]
<<<<<

Constraints:
- The changes should remain simple within the size of a learning project
- The backend must still use Python, FastAPI, and Pydantic for validation
- Keep the existing architecture whenever it fits
- Keep the tech stack simple, well-documented, and easy to run locally
- No authentication or multi-tenancy

- Do not suggest microservices, Docker, cloud deployment, or production database setup

Task: Propose two reviewed lightweight architectures:
- Option A should be the simplest local-storage approach with minimal changes to the existing architecture.
- Option B may use a lightweight local database approach if it improves realism without overcomplicating the project.
Output: Generate a mini-ADR as an addendum to the existing architecture, which can be copied into a new Word document

AI Response

See attached ADR-001 Addendum: Due Date, Overdue Logic, and Advanced Filtering

Prompt for Comparing original tracker ADR and MINI-ADR

AI Tool Used: ChatGPT Pro
Model: GPT 5.5

Task: Compare the two and tell me the differences in their functional and technical specifications.

Constraints:
- Do not compare word by word basis

Output: Present comparison in a table form

AI Response / Inspection and Review

After the comparison is generated, each row was inspected to check consistency to either accept or update/reject.

Area	ADR-002: Existing Baseline Architecture	ADR-001 Addendum: Due Date & Filtering Extension	Functional Specification Difference	Technical Specification Difference	Decision
Document purpose	Defines the original Task Tracker architecture for Module 1.	Defines an addendum for extending the existing application.	ADR-002 covers the original application scope. The addendum covers new due-date-related features.	The addendum keeps the same architecture but specifies additional changes to the model, validation, service, frontend, and tests.	Accepted
Architecture decision	Selects **Option A: FastAPI with JSON File Storage**.	Keeps **Option A: FastAPI with local JSON file storage** as the recommended architecture.	No functional change in architecture choice.	No major architecture change.The existing architecture remains suitable.	Accepted

Area	ADR-002: Existing Baseline Architecture	ADR-001 Addendum: Due Date & Filtering Extension	Functional Specification Difference	Technical Specification Difference	Decision
Core functionality	Create tasks, view tasks, filter by status/priority, update details, update status using a forward-only workflow, delete tasks, and persist tasks.	Keeps those features and adds due date, due date change tracking, overdue detection, and wider filtering/search.	The addendum expands the feature set beyond basic task tracking.	Existing modules must be extended, not replaced.	Accepted
Task data model	Task fields are: id, title, description, status, priority, and assignee.	Adds due_date, due_date_change_date, and computed overdue.	Tasks now carry scheduling information.	Backend model/schema/storage must support new fields. Overdue is response-only and not stored.	Accepted
Due date	No due date field is specified.	Each task must have a required due_date.	New functional requirement: every new task must have a due date.	Pydantic's create schema must make due_date required and validate it as a date.	Accepted
Due date change date	Not included.	Adds due_date_change_date.	Users can see whether the due date was changed after creation.	The service layer must set this automatically when due_date changes. It should be stored because it records a historical change.	Accepted
Overdue logic	Not included.	A task is overdue when due_date < current_server_date and status is ToDo or InProgress.	New functional behavior: users can identify overdue tasks.	Overdue must be calculated dynamically by the service layer and not saved permanently in tasks.json.	Accepted
Done task behavior	Done is already a terminal status.	Done tasks are never overdue, even if the due date has passed.	Adds a functional rule that connects status to overdue behavior.	Service logic must force overdue = false for Done tasks.	Accepted
Filtering	Supports filtering tasks by status and priority.	Expands filtering to due date, overdue flag, status, priority, assignee, and task name.	Users can locate tasks using more criteria.	Filtering logic must support additional query parameters and combine them with a logical AND.	Accepted

Area	ADR-002: Existing Baseline Architecture	ADR-001 Addendum: Due Date & Filtering Extension	Functional Specification Difference	Technical Specification Difference	Decision
Search by task name	Not specified as a search feature.	Adds partial, case-insensitive search by task title/name.	New functional search capability.	The service layer must implement normalized title comparison, likely using lowercase matching.	Accepted
Validation	Validates required fields, title length, allowed status, allowed priority, whitespace-only title rejection, and required assignee.	Adds validation for a required due_date, a valid date format, and blocks user-supplied due_date_change_date or overdue dates.	New user-facing validation errors for missing/invalid due dates.	Pydantic schemas must distinguish client-input fields from system-generated /response fields.	Accepted
System generated fields	ID is generated by the system.	due_date_change_date is also system-generated; overdue is computed.	More fields are controlled by the backend rather than the user.	The backend must prevent manual client control over due_date_change_date and overdue.	Accepted
ID type	Specifies id as int or UUID.	Suggests id as string or UUID.	No direct functional change for users if IDs are opaque.	Potential technical inconsistency: the addendum suggests UUID/string, while ADR-002 specifies an integer ID. This should be clarified before implementation.	UUID will be used
Service layer Role	Create, update, delete, filter, enforce title uniqueness, enforce status transitions, generate IDs, coordinate storage.	Adds due date update logic, due date change tracking, overdue calculation, extended filtering, and title search.	The service layer now handles scheduling and derived-status behavior.	The service layer becomes more important and may need helper functions for date logic.	Accepted
Storage layer	Reads and writes tasks to backend/data/tasks.json.	Still uses JSON storage, but stored records must include due_date and due_date_change_date.	Stored task data becomes richer.	JSON read/write structure must be updated, and legacy records without due_date may need normalization.	Accepted

Area	ADR-002: Existing Baseline Architecture	ADR-001 Addendum: Due Date & Filtering Extension	Functional Specification Difference	Technical Specification Difference	Decision
Computed vs stored data	No computed response field is specified.	Overdue is computed and included in responses but not stored.	Users see the overdue status without manually setting it.	Requires a response schema or serialization logic to add overdue dynamically.	Accepted
Frontend behavior	Displays tasks, collects user input, calls REST endpoints, and renders responses.	Adds due date input, due date display, due date change date display, overdue display, and new filter controls.	The frontend becomes capable of schedule-based task viewing.	The frontend should display the backend-calculated overdue status but should not implement the core overdue rule.	Accepted
Testing scope	Tests include create, view, filter, update, status, delete, and persistence.	Adds tests for due date creation, missing due date, invalid date, due date update, overdue calculation, filtering, search, and combined filters.	More acceptance scenarios must be covered.	New test files are proposed: test_due_date.py, test_due_date_change.py, test_overdue.py, and test_search_filter.py.	Accepted
Optional utility module	No date utility module is specified.	Suggests optional utils/date_utils.py.	No direct user-facing change.	Technical improvement to avoid making the service layer too large with date parsing/comparison logic.	Accepted
Alternative Option B	Option B is SQLite with SQLAlchemy.	Option B is SQLite with SQLModel.	No immediate functional difference because Option B is not selected in either document.	Technical inconsistency: SQLAlchemy vs SQLModel should be aligned if Option B is documented for future use.	Alternative B is rejected so this recommendation can be kept in ADR-001 addendum.

Area	ADR-002: Existing Baseline Architecture	ADR-001 Addendum: Due Date & Filtering Extension	Functional Specification Difference	Technical Specification Difference	Decision
Risks	Focuses on JSON limitations, full-file rewrites, concurrency limitations, and production unsuitability.	Adds risks around due-date conflicts, field naming, computed overdue storage, client-supplied system fields, date comparison, and legacy JSON records.	The addendum identifies risks caused by the new features.	More implementation-specific risk controls are needed for date handling and filtering.	Accepted
Scope	Covers the original Module 1 learning-project architecture.	Covers only the extension for due date, due date change date, overdue calculation, due date update, filtering, and searching.	The addendum is narrower but feature-specific.	It should be treated as an extension of the baseline ADR, not a replacement for it.	Accepted
Implementation impact	Establishes the base backend/frontend/storage structure.	Requires updates to the model, schemas, service logic, JSON data, frontend display, filters, and tests.	The application moves from simple CRUD task tracking to schedule-aware task tracking.	Implementation remains lightweight but touches almost every layer of the existing application.	Accepted

Storage Scheme

In several areas of the mini-ADR (ADR 001-Addendum), an inconsistency was identified between the addendum's storage assumption and the original running task-tracker storage scheme.

The generated mini-ADR treated Option A as FastAPI with **persistent** local JSON file storage (`tasks.json`), including a JSON repository that reads and writes to that file across restarts. The already implemented Task Tracker saves tasks in a JSON file, but the original tracker does not keep them after restart.

The approved user stories (US-08 to US-11) require a due date, a due date change date, overdue detection, and filtering on the shared task list; they do not require data to persist after restarts.

Though keeping a local JSON file is essential to avoid having the user recreate the tasks after each restart, in this addendum we used in-memory storage and parked the requirement for persistent JSON storage for the next sprint. It will not be implemented in this specific extension project.

Prompt for Generating the Mini-ADR Documentation

You are working inside an existing Task Tracker web application project.

Your task is to create a Markdown file in the project `docs` folder using the mini-ADR text provided below.

[PASTE THE ADR 001-ADDENDUM REVIEWD AND APPROVED FOR THE EXTENDED FEATURES]

AI Response

See project folder /task-tracker - mid/docs/mini_adr.md

Backend Implementation

Prompt for Backend Implementation

AI Tool Used: AI within Cursor IDE
Model: Cursor Grok 4.6 High

Role: You are a senior Python backend engineer.

Task: Review @docs/user_stories.md @docs/mini-adr.md @backend/app/main.py @backend/app/models.py and generate a plan to implement the additional features to the backend-only.

Constraint:
- Do not apply code changes yet

Output:
- present the changes as a list of tasks presented in a table form

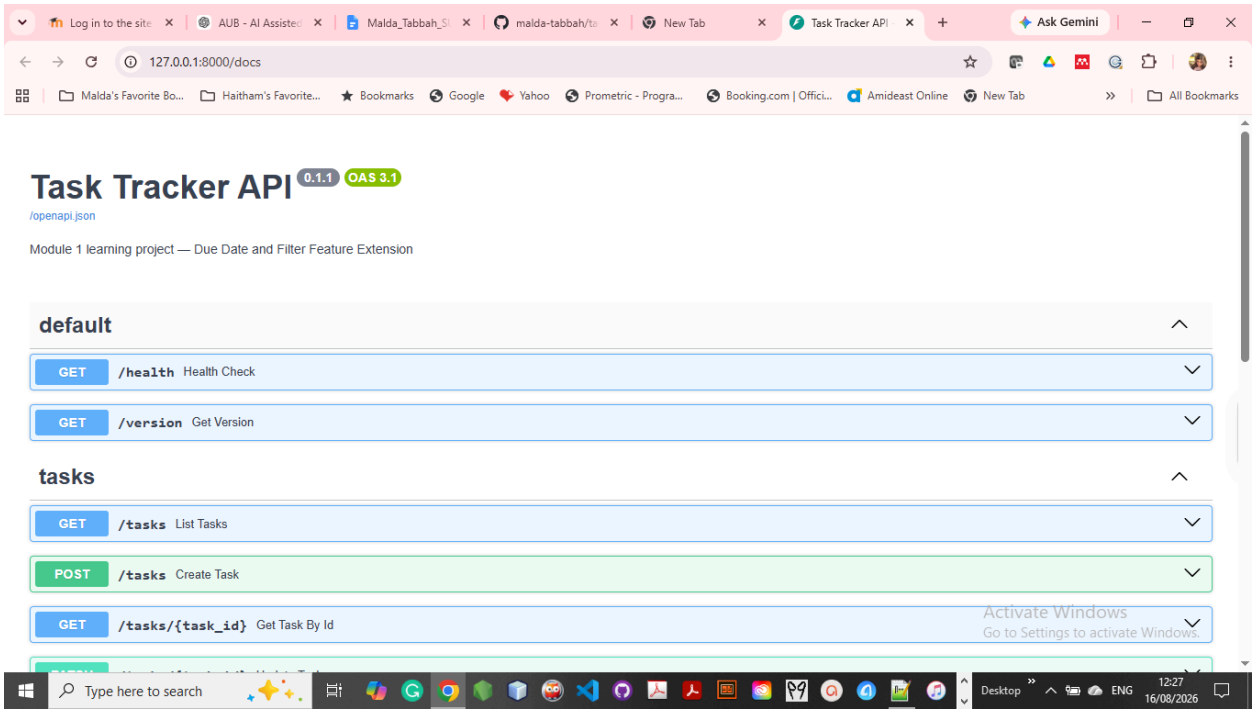
AI Response

Task	Status
Extend TaskCreate, TaskUpdate, TaskResponse with due_date, due_date_change_date, overdue	Accepted
Add is_overdue() in business_rules.py using server date and status	Accepted
Store due_date fields; recompute overdue on every add/get/list/update return	Accepted
On PATCH, set due_date_change_date only when due_date changes actually	Accepted
Add GET /tasks query params: assignee, due_date, overdue, title (AND with existing filters)	Accepted
Update existing tests to send due_date and expect new response fields	Accepted
Add pytest coverage for US-08–US-11 (due date, change date, overdue, combined search)	Accepted

Backend Verification

The backend is running at http://127.0.0.1:8000.

- Health: http://127.0.0.1:8000/health
- Swagger: http://127.0.0.1:8000/docs



Weak Prompt

I want to test all the backend points and make sure all user stories acceptance criteria are met. Please prepare the test script

Result of Weak Prompt

The model created acceptance_test.py in the tests folder, which I did not want.

Prompt for Document Manual Verification using Curl

Role: I am senior backend I want to verify that all acceptance criteria listed in @docs/user_stories.md are satisfied

Task: Generate a list of curl commads that can i execute on the terminal to verify all acceptance criteria

Constraint:
- Do not run the curl test. Let me do them one by one

Output
- The list should be in table form and have the following columns: user story id, acceptance criteria, curl command that should be run, and the expected response

Response AI
See project folder /task-tracker - mid/docs/backend_verification.md

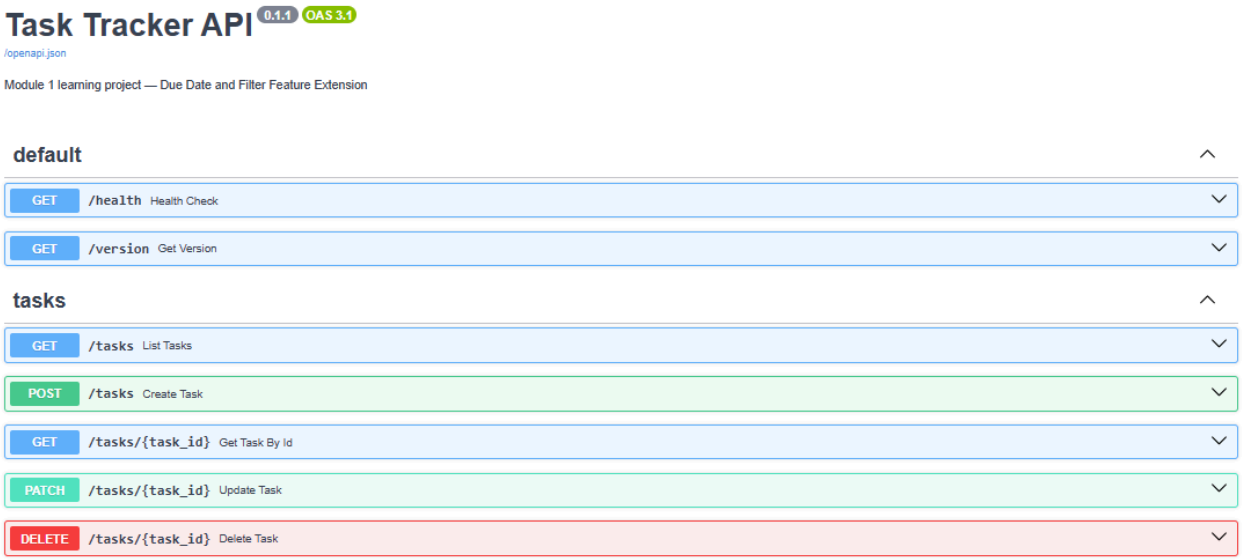
Follow-up Prompt for running tests documented for the extended features

Context: Now I want to run all tests in:
@backend/tests/test_overdue.py
@backend/tests/test_overdue.py
@backend/tests/test_search_filter.py

Task:
- Run the test one by one
- Log the results of each test
- Write the results in a new Markdown file called verification.md under docs/

Output: Document test results in a table form

Start the backend
Run Swagger UI
http://localhost:8000/docs



Frontend Implementation

Prompt for Frontend Implementation

AI Tool Used: AI within Cursor IDE
Model: Cursor Grok 4.6 High

Role: You are a senior Python frontend engineer.

Task: Review @docs/user_stories.md @docs/mini-adr.md @backend/app/main.py @backend/app/models.py and @frontend/index.html and generate a plan to implement the additional features in the frontend.

Constraint:
- Do not apply code changes

Output:
- present the changes as a list of tasks presented in table form

Frontend Verification

The frontend is running at http://127.0.0.1:5500/index.html.

The API is still at http://127.0.0.1:8000, which is the origin the page calls. Open that frontend URL in the browser to use the board.

Frontend Verification Script

Area	What was done	What was observed	Figure Reference
Empty board	Load the page with no tasks	Three columns; “Drop tasks here”; not an error card	Figure 1
Create (US-08)	Add a task with title + due date	Card appears in ToDo with Due YYYY-MM-DD; no “Changed” date	Figure 2 and 3
Missing due date	Submit create without a due date	Form error; task is not created	Figure 4
View due date / overdue (US-08, US-10)	Create one past due (2026-08-01) and one due today	Past-due ToDo/InProgress shows overdue; due today does not; Done never shows overdue	Figure 5
Update due date (US-09)	Edit a task’s due date	Due date updates; Changed shows today’s date (2026-08-16)	Figure 6
Complete overdue (US-10)	Drag ToDo → InProgress → Done	Overdue disappears when status is Done	Figure 7
Invalid drop	Drag ToDo onto Done	“Invalid status transition.”; card stays in ToDo	Figure 8
Return of Overdue flat	Drag Overdue Task from Done to In Progress	Board shows the overdue task in the progress column and the overdue flag is displayed	Figure 9
Filters (US-11)	Filter overdue	Board shows only matches (AND if several filters are set)	Figure 10 and 11
No matches (US-11)	Filter for something that does not exist	Empty columns plus “No tasks match the selected filters.”	

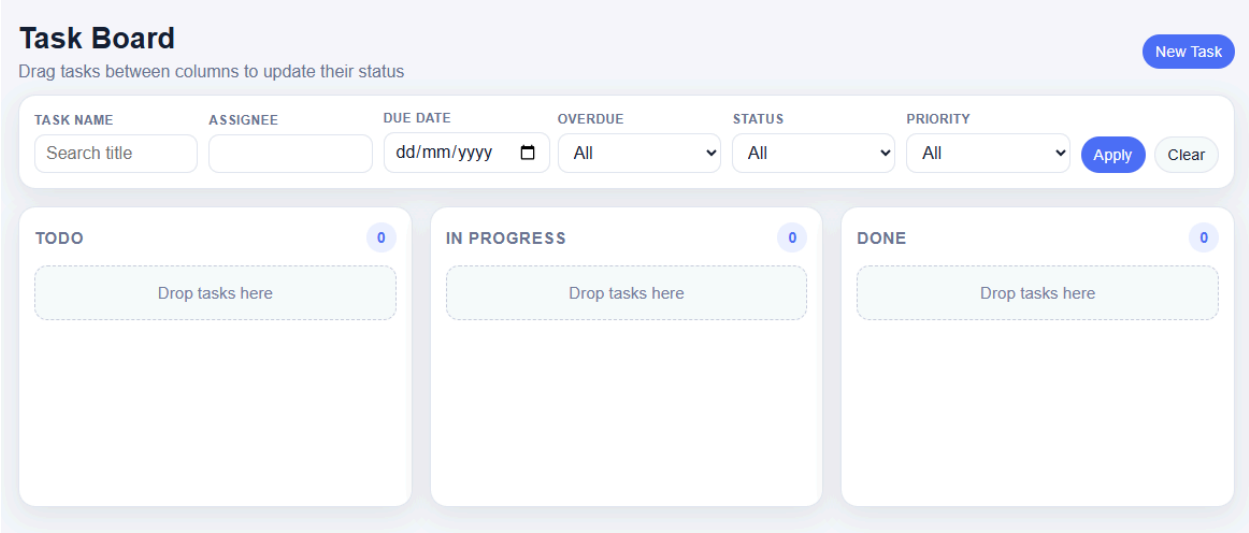


Figure 1 - Empty Board

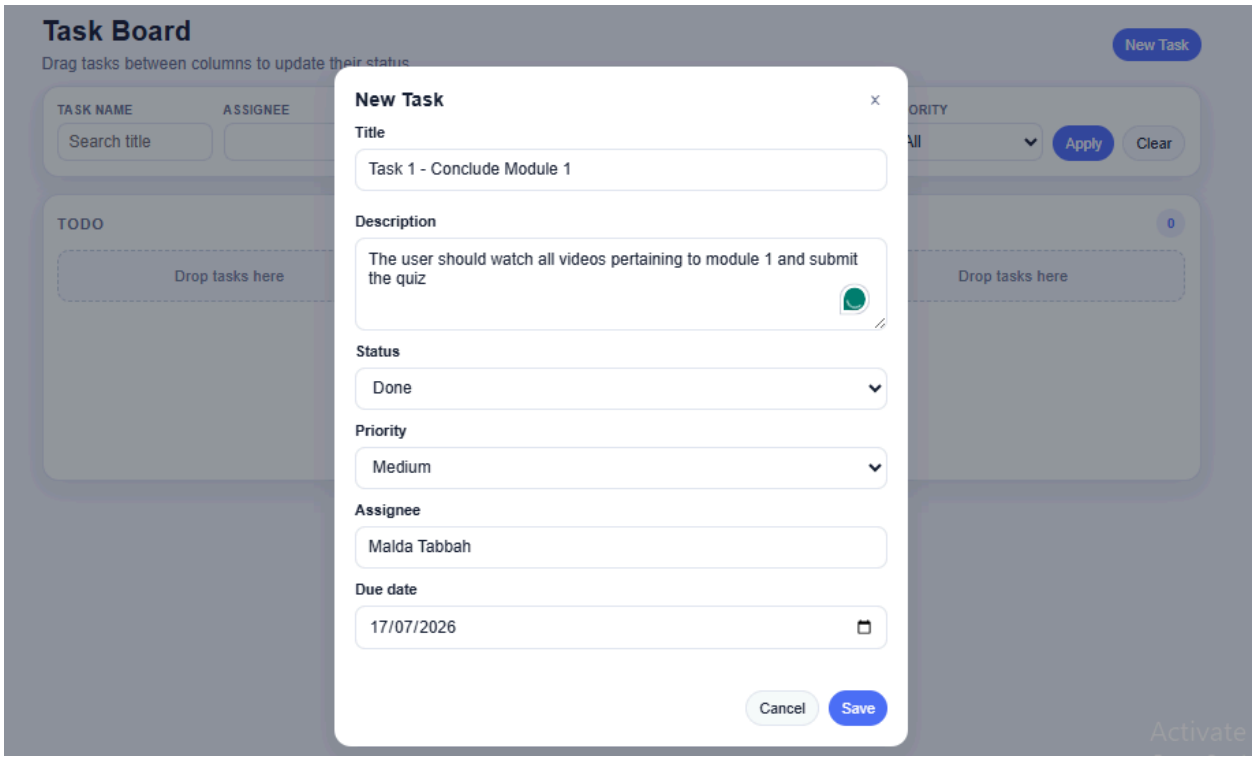


Figure 2 - Adding a new Task

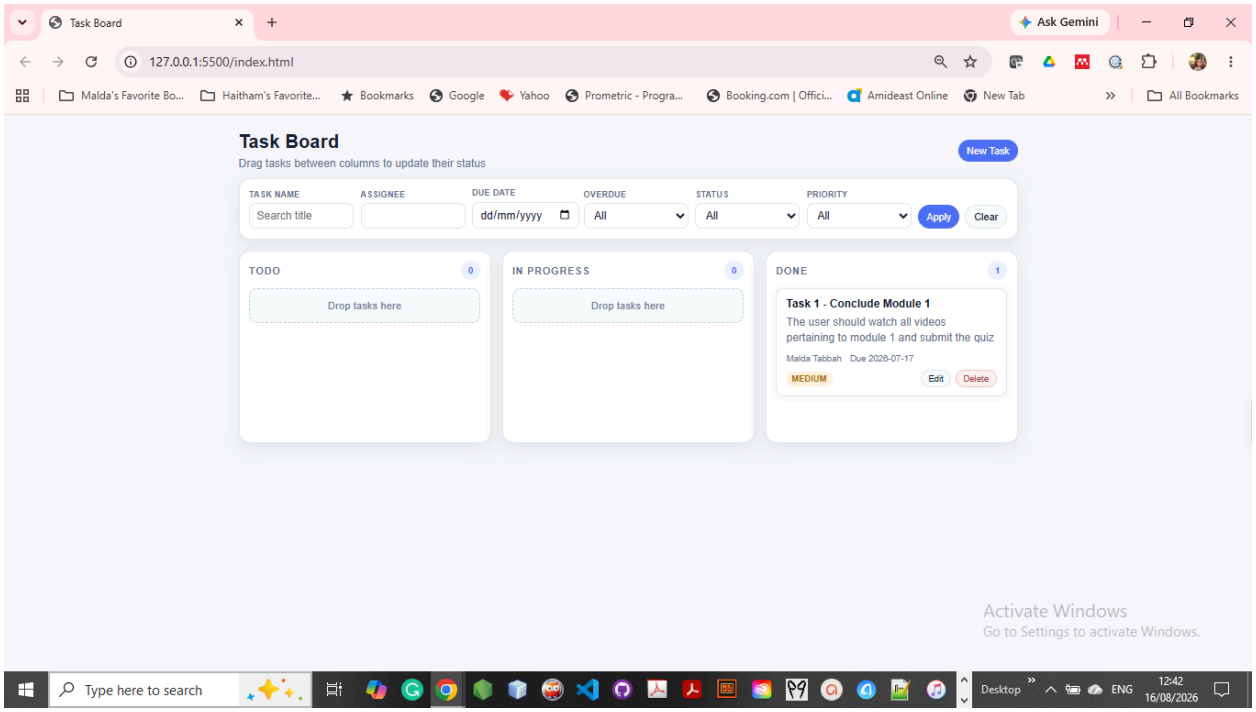


Figure 3 - After Saving

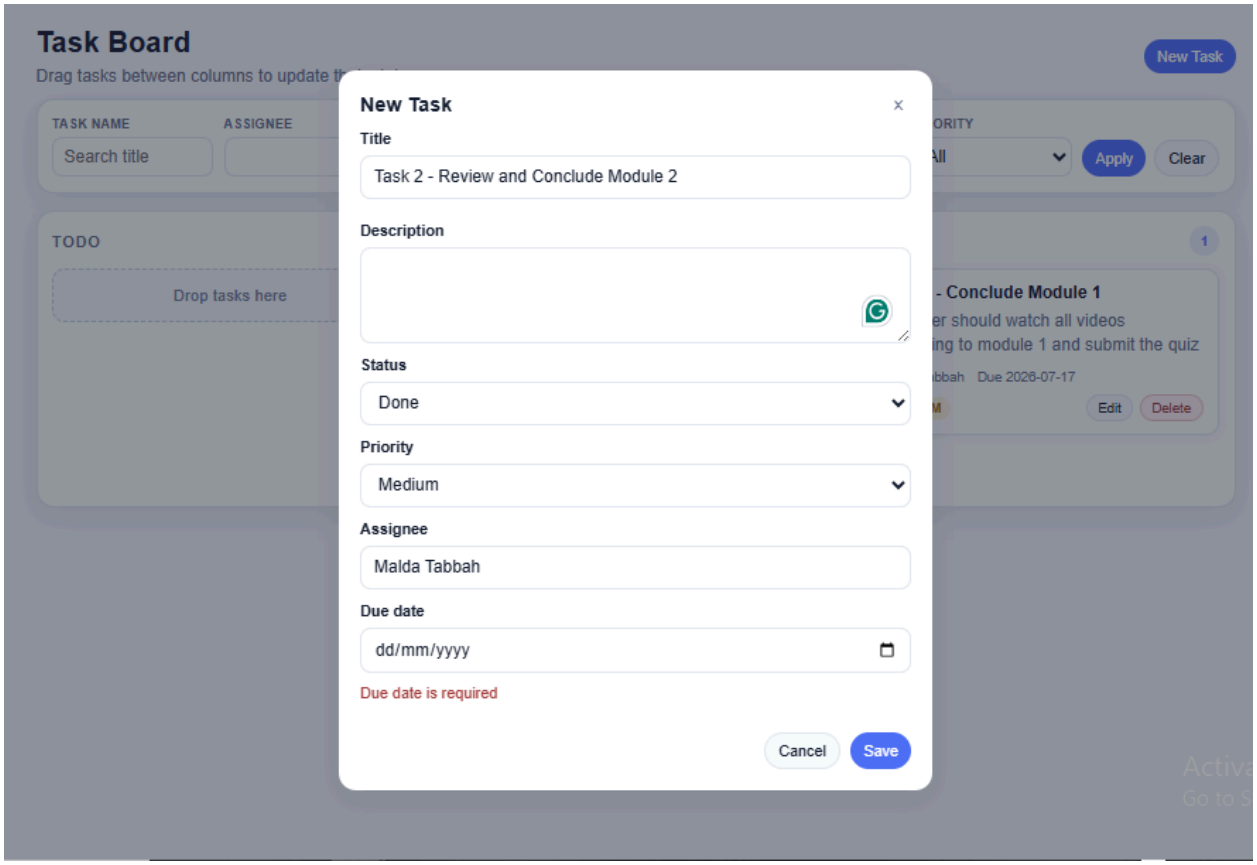


Figure 4 - Date is Required

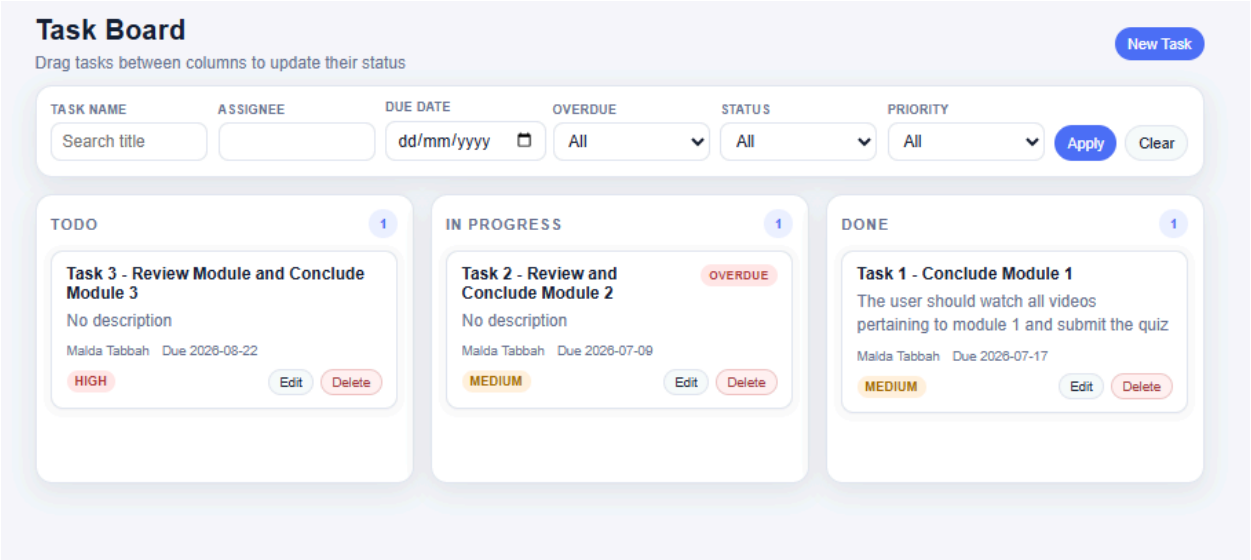


Figure 5 - Due Tasks in progress, Done Tasks and Overdue Tasks

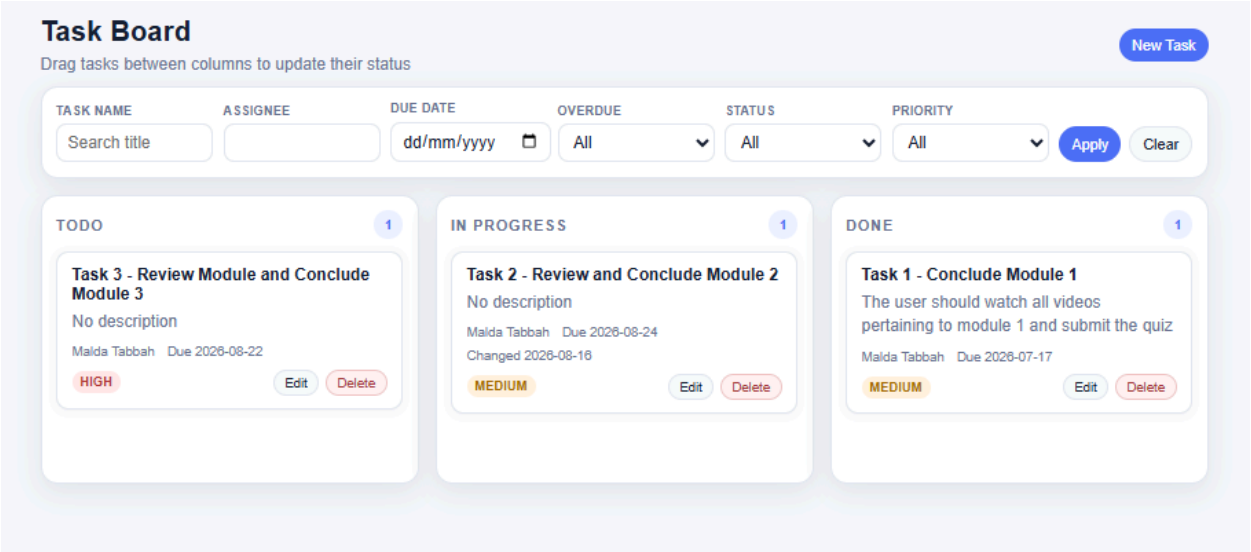


Figure 6 - Overdue Task has its due date changed

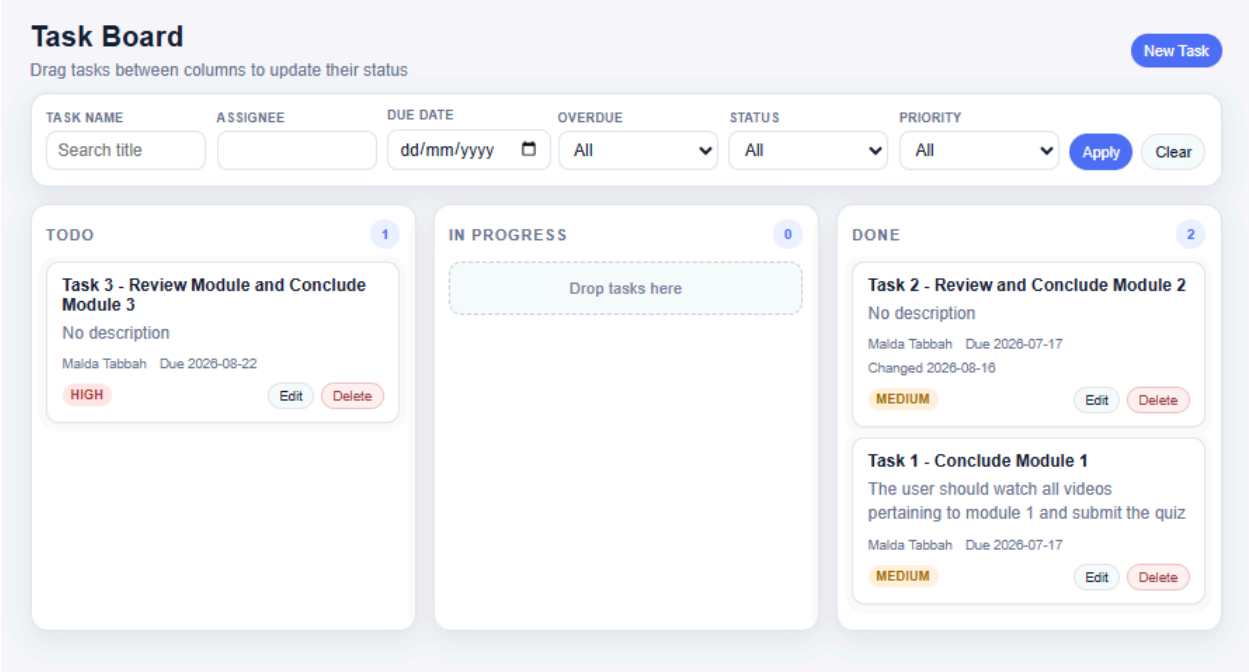


Figure 7 - Overdue flag no more shows when the task is done

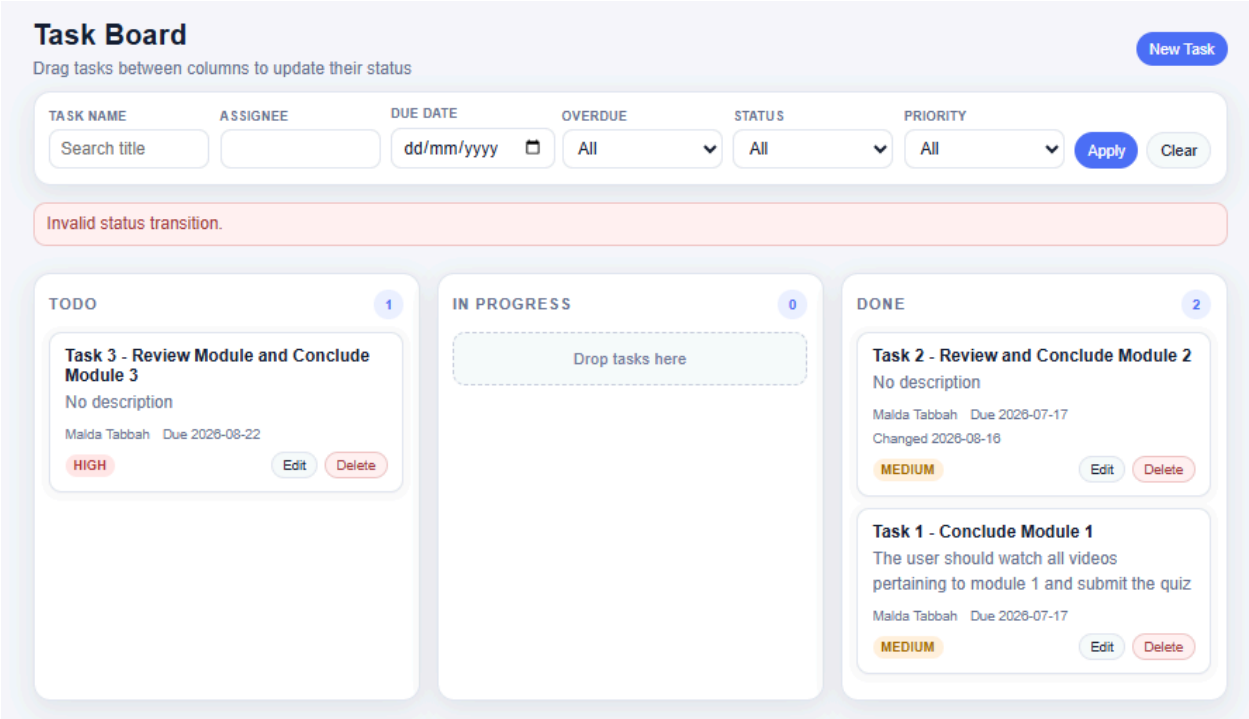


Figure 8 - Dragging ToDo Task to Done, Done Task to ToDo

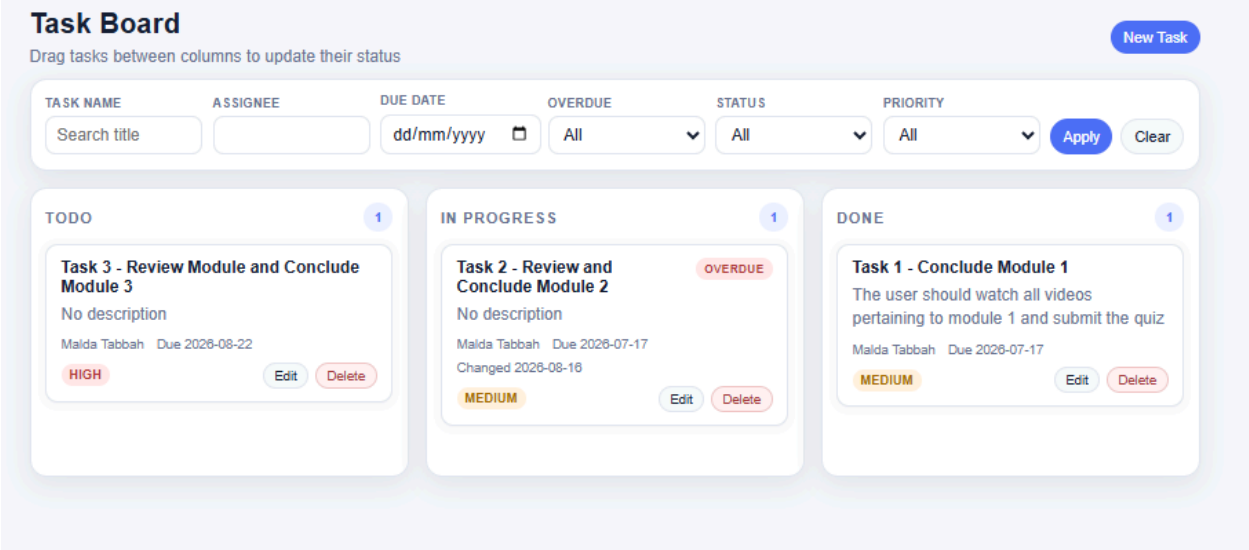


Figure 9 - Dragging overdue tasks from Done to In Progress shows the overdue flag

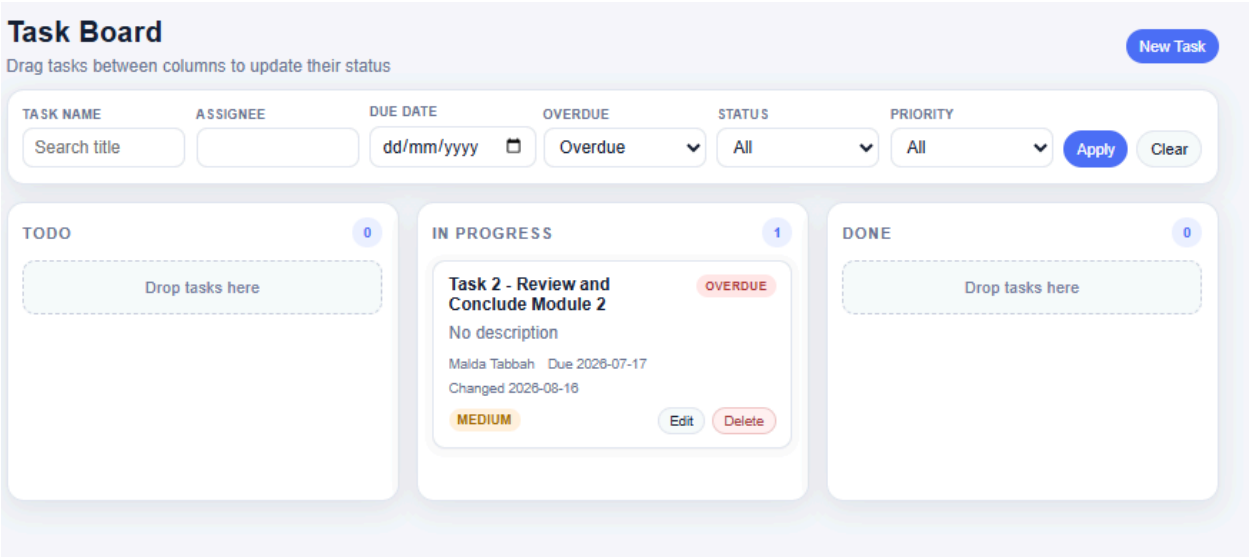


Figure 10 - Filter by Overdue Date

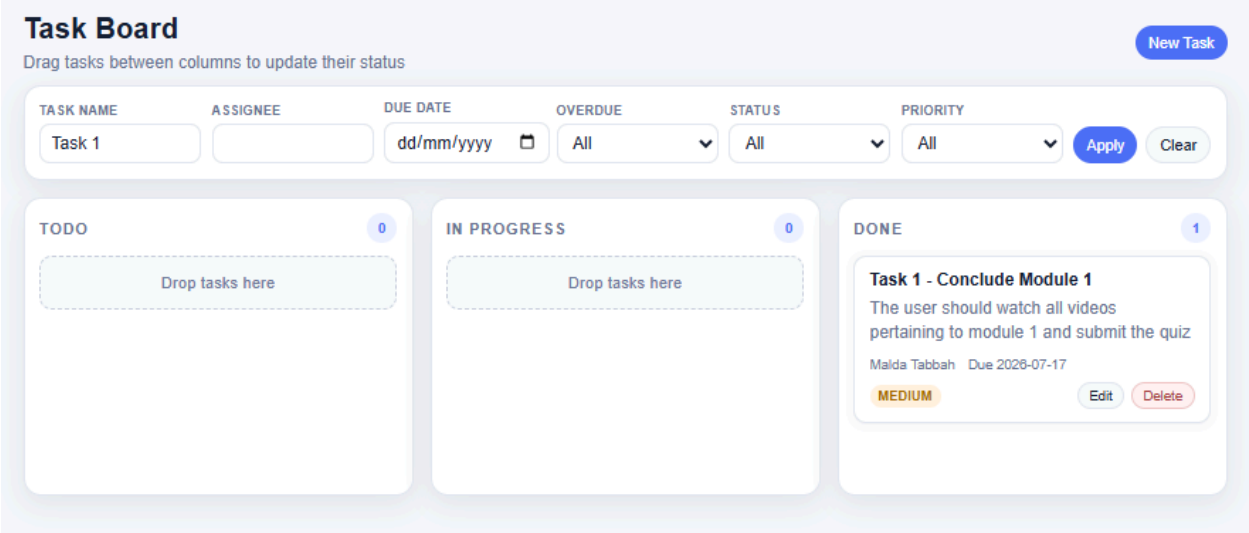


Figure 11 - Search Task Name

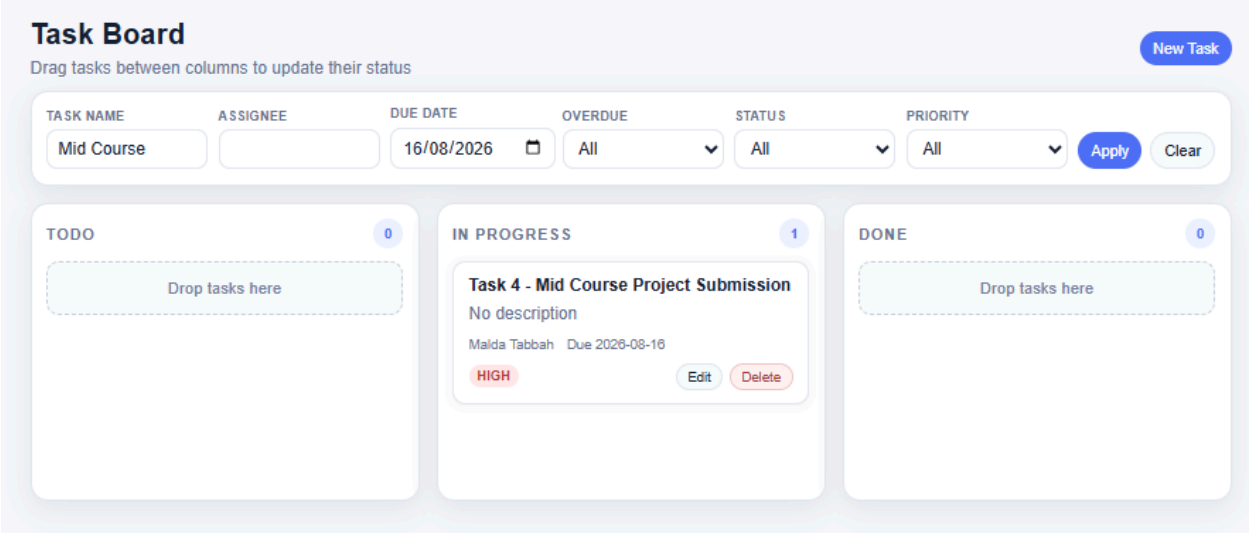


Figure 11 - Multi-Criteria Search (Name and Due Date)

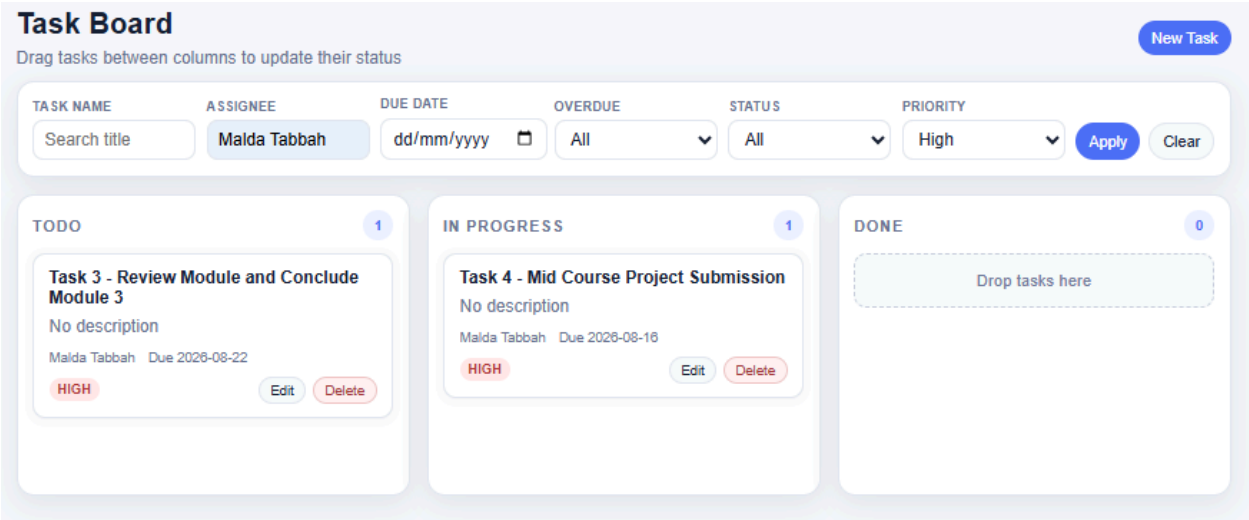


Figure 12 - Multi-Criteria Search (Assignee and Priority)

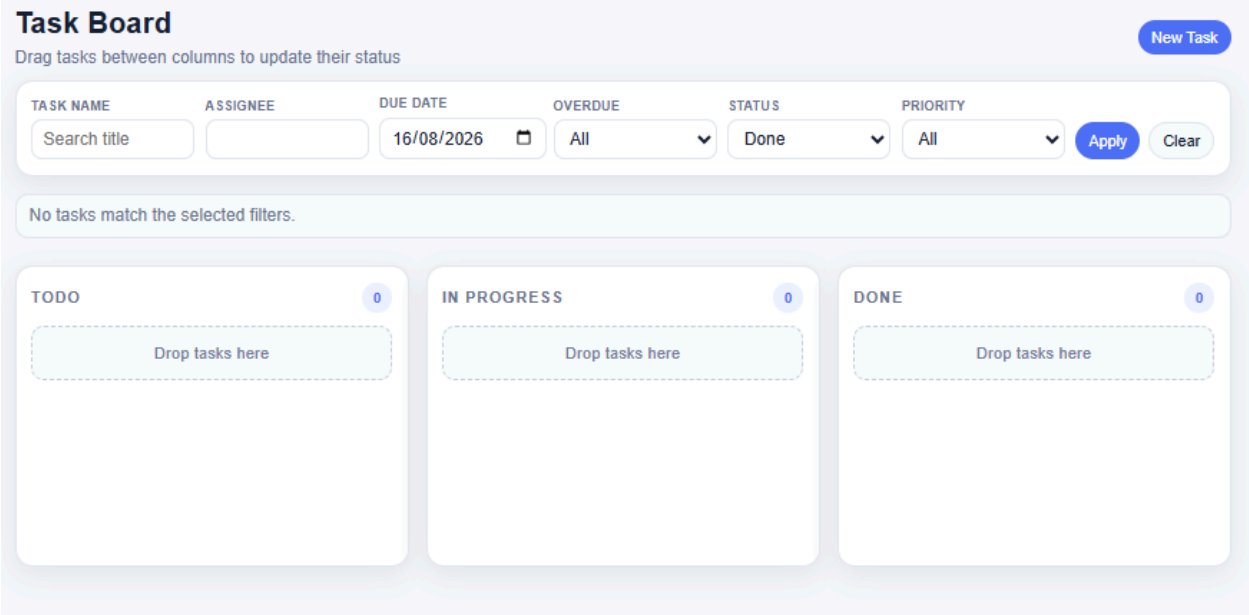


Figure 13 - No tasks matching search criteria